

Tarea 2 – Autenticación multifactor y Zero Trust

Diplomatura en Ciberseguridad – UNC

Integrantes: Asael Jara, Santiago Pustilnik, Diego Oleiarz

1. OTP y su implementación

1.1. ¿Qué es un OTP?

Un OTP (One-Time Password) es una **contraseña** que solo se puede usar una vez y que suele tener una validez corta en el tiempo.

Se usa como segundo factor de autenticación en un esquema de MFA (Multi-Factor Authentication).

Normalmente se genera con algoritmos como TOTP (Time-based One-Time Password) o HOTP (HMAC-based One-Time Password), definidos en estándares como RFC 6238 y RFC 4226.

En la práctica, el usuario algo que sabe (la contraseña) más algo que tiene (el teléfono con la app de OTP). Esto reduce el riesgo de acceso no autorizado si la contraseña se filtra.

1.2. Uso de Google Auth con GitHub

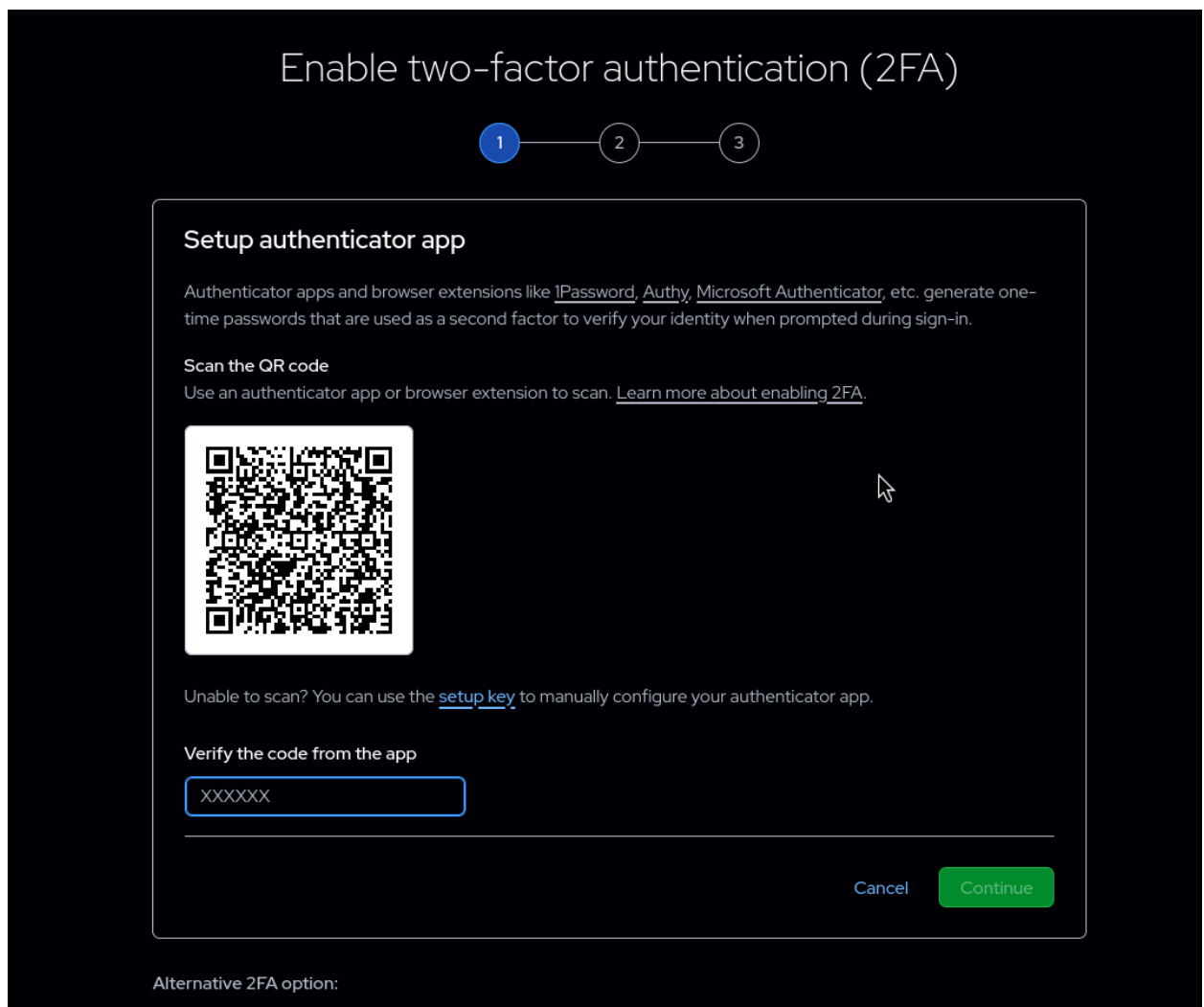
Utilizamos Google Auth como aplicación de OTP y GitHub como servicio de terceros donde habilitamos el segundo factor de autenticación.

Requerimientos para la implementación

- Cuenta en GitHub
- Celular
- Aplicación OTP

Pasos realizados

1. Iniciar sesión en GitHub con usuario y contraseña.
2. Ir a:
Settings → Password and authentication → "Two-factor authentication (2FA)".



3. Abrir Google Auth en el teléfono y agregar una nueva cuenta:
 - Escanear el código QR que muestra GitHub
4. Verificar que en Google Auth aparece un nuevo código para GitHub, este cambia cada cierto tiempo.
5. Volver a GitHub y escribir el código de 6 dígitos generado por Google Auth para completar la configuración.
6. GitHub muestra los códigos de recuperación; lo mejor es guardarlos en un gestor de contraseñas, en un archivo cifrado o escribirlos en un papel.
7. Cerrar sesión y volver a iniciar sesión en GitHub para probar el 2FA:
 - Ingresar usuario y contraseña.
 - Abrir Google Auth y escribir el código que aparece en el login.

1.3. Opinión sobre Google Authenticator

Ventajas:

- Es fácil de usar, la interfaz es simple: muestra una lista de cuentas con códigos que se actualizan cada 30 segundos
- Es compatible con la mayoría de los servicios que usan TOTP estándar (GitHub, Google, Facebook, etc.).

- Está disponible para Android y iOS.
- Permite hacer copia de seguridad de los códigos sincronizándolos con la cuenta de Google, lo que ayuda si se pierde o se cambia de dispositivo.
- Se puede bloquear la app con PIN o biometría para que nadie use los códigos si tiene acceso físico al teléfono.

Desventajas:

- A diferencia de algunas alternativas, no es software libre, por lo que no se puede auditar el código.
 - Si alguien logra comprometer la cuenta de Google del usuario y la sincronización está activada, podría obtener los OTP almacenados en la nube.
 - No está tan orientado a entornos empresariales con funciones avanzadas (como auditoría detallada o gestión de equipos), es más una herramienta básica de usuario final.
-

2. Zero Trust y AAA en entornos cloud

2.1. Principios de Zero Trust asociados a AAA

Zero Trust se basa en la idea de “nunca confiar, siempre verificar”.

- **Autenticación**: se debe autenticar no solo al usuario, sino también a cada servicio o microservicio, usando identidades fuertes y credenciales de corta duración.
- **Autorización**: las decisiones se toman por solicitud (per-request) y siguiendo mínimo privilegio, usando políticas que combinan identidad de usuario, identidad de servicio y atributos de contexto (ubicación, tipo de recurso, etc.).
- **Accounting**: la arquitectura requiere telemetría y monitoreo continuo de accesos y flujos de servicios para ajustar políticas, detectar anomalías y verificar que usuarios y recursos se comportan de acuerdo a su rol.

En vez de segmentar solo por IP, VLAN o subred, Zero Trust propone una **segmentación basada en identidades** (identity-based segmentation), donde las políticas se expresan sobre “quién es el servicio/usuario” y qué operación intenta realizar.

2.2. Aspectos en un entorno cloud

En un entorno cloud, especialmente cloud-native y con aplicaciones distribuidas, hay "complicaciones" con zero trust en:

- Los recursos pueden estar repartidos en múltiples nubes y ubicaciones (on-premise, nubes públicas).
- El perímetro tradicional desaparece; hay servicios expuestos mediante APIs, microservicios, contenedores, etc.
- Es necesario proteger tanto el tráfico cliente <-> aplicación como servicio <-> servicio dentro del cluster.
- La identidad ya no es solo el usuario, también abarca servicios, workloads, contenedores y dispositivos.

2.3. Requerimientos de ZTA para un control de acceso granular

ID-SEG-REC-1 – Conexión cifrada entre servicios	toda comunicación entre servicios, sin importar la nube o red, debe estar cifrada (por ejemplo, con mTLS) para proteger contra escucha y falsificación.-
ID-SEG-REC-2 – Autenticación de servicios	cada servicio presenta credenciales criptográficas de corta duración, verificables, y se autentica por conexión, con re-autenticación periódica según la validez del certificado.
ID-SEG-REC-3 – Autorización servicio a servicio	se usan las identidades de servicio en tiempo de ejecución para aplicar políticas granulares; los proxies pueden consultar motores externos de autorización si necesitan decisiones más dinámicas.
ID-SEG-REC-4 – Autenticación de usuario final	se requiere un sistema de gestión de identidades que emita tokens de usuario (por ejemplo JWT u OAuth2) y aplique MFA resistente a phishing, autenticando el token en cada salto de la cadena de microservicios.
ID-SEG-REC-5 – Autorización usuario-recurso	cada petición verifica que el usuario autenticado está autorizado para el recurso y la operación, ya sea localmente contra claims del token o mediante un PDP externo.

Además, el modelo introduce **políticas multi-tier**: combinar políticas de red (network-tier, como reglas de firewall y microsegmentación) con políticas de identidad (identity-tier) para lograr un control granular sin depender solo de IPs y puertos

2.4. Despliegue de ZTA

El despliegue de una arquitectura Zero Trust en entornos cloud-native y multi-location suele incluir:

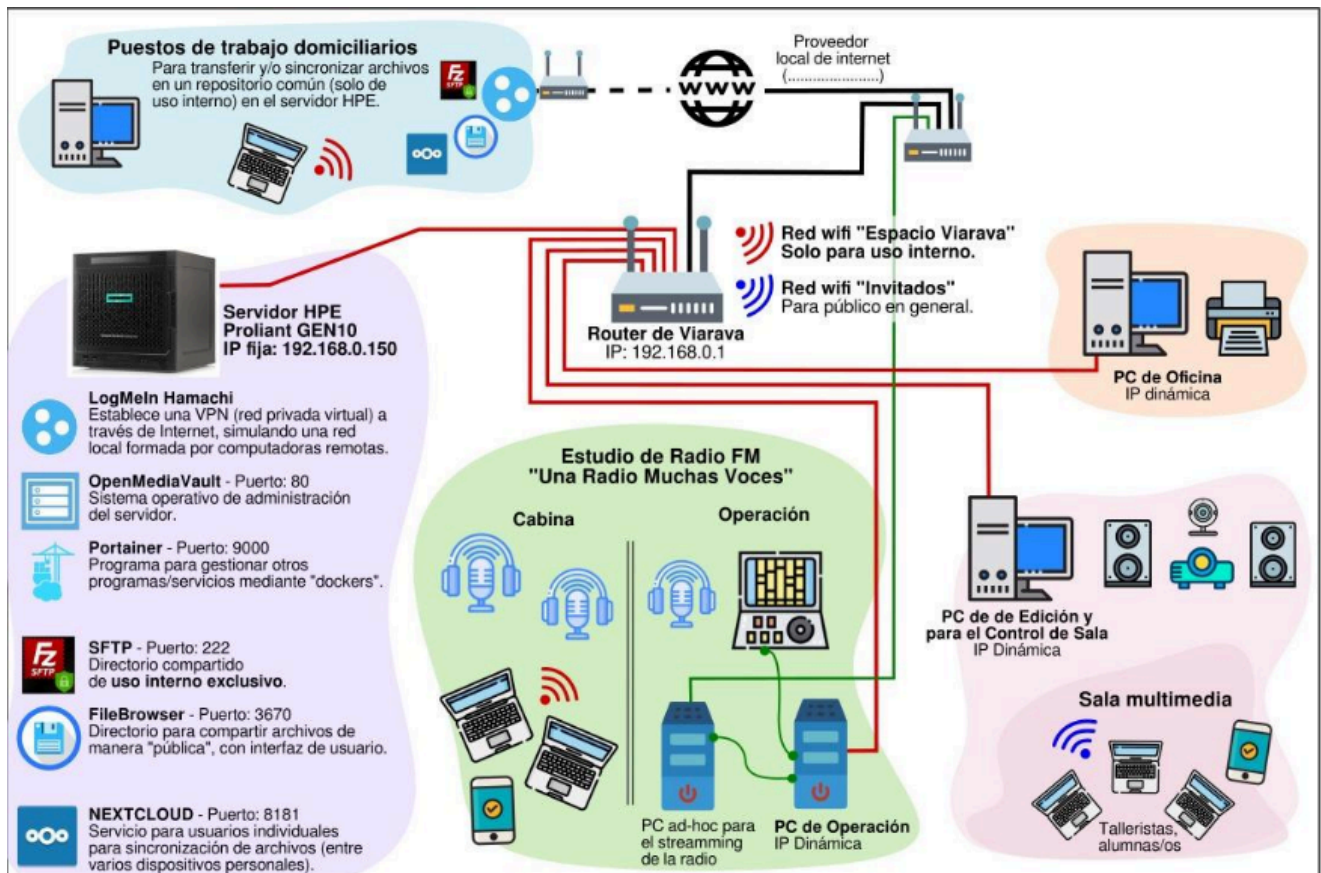
- Uso de un **service mesh** (por ejemplo, Istio, Linkerd) para gestionar el tráfico entre servicios con mTLS y políticas centralizadas.

- Un sistema de identidad para workloads (por ejemplo, SPIFFE/SPIRE) que emite identidades a los servicios.
- Un IdP para usuarios (por ejemplo, integrado con SSO corporativo) que emite tokens de acceso.
- PEPs ubicados en:
 - Proxies de entrada (API gateways, ingress controllers).
 - Sidecars del service mesh para controlar tráfico servicio a servicio.
- Un PDP que evalúa políticas de acceso (puede ser un motor de políticas como OPA – Open Policy Agent).
- Integración con sistemas de logging, SIEM y monitoreo para el accounting y la detección de anomalías.

2.5. Análisis de la política basada en identidades

La política basada en identidades cambia el foco desde la red a los sujetos (usuarios y servicios):

- Las reglas ya no se expresan como “la IP X puede hablar con la IP Y”, sino como “el servicio front-end puede invocar al servicio back-end usando tal metodo y tal ruta”.
- Las **identidades de servicio** se representan con credenciales criptográficas (por ejemplo, SPIFFE IDs contenidos en certificados X.509), lo que permite autenticación mutua y autorizaciones consistentes sin depender de la topología de red.
- Estas políticas son “policy-agnostic”: si un microservicio se mueve de un cluster on-prem a una nube pública, la política sigue siendo válida porque “sigue” a la identidad de la aplicación y no a la IP o VLAN.
- Al ser expresables como “policy as code”, pueden integrarse en pipelines CI/CD y probarse automáticamente, aumentando la precisión y reduciendo errores operativos.



1. Router perimetral

a. *Control*: Firewall de perímetro + VPN para acceso remoto

- **Riesgo mitigado**: Acceso no autorizado desde Internet, ataques DDoS, explotación de servicios expuestos
- **Tipo**: ACL (Access Control Lists) + IPSec/OpenVPN

INTERNET <-> Router Variante <-> Red Interna
 ↓
 REGLAS DE FIREWALL

b. *Viabilidad Economica*

- El router ya lo tiene (probablemente soporta ACL básicas)
- Costo: Configuración interna. Contratar personal IT
- Fin: Protege toda la red interna

PCs Operación/Administración (con acceso al servidor)

a. *Control*: GPO (Group Policy Objects) + Endpoint Protection + DLP (Data Loss Prevention)

- **Riesgo mitigado:** Malware lateral, robo de credenciales admin, exfiltración de datos sensibles
- **Tipo:** Políticas centralizadas + antivirus + prevención pérdida datos
 - b. Viabilidad económica
- Native en Windows Domain
- Windows Defender + Microsoft Defender for Endpoint
- Implementación: 1 día aprox